

Database Schema — Black Fitness ระบบจองคลาส

เอกสารนี้อธิบายโครงสร้างฐานข้อมูล Cloud Firestore ของระบบ Black Fitness ครอบคลุมทุก collection ฟิลด์ ชนิดข้อมูล ความหมาย ความสัมพันธ์ และ index ที่จำเป็น

อ้างอิงจากซอร์สโค้ดจริง — ฟิลด์ทุกตัวมาจากการอ่าน/เขียนใน Vue components, stores, และ scripts ฟิลด์ที่ไม่ได้ใช้ในโค้ดจะไม่ถูกระบุ (Firestore เป็น schemaless จึงเป็น schema เชิงพฤกษกรรม)

สารบัญ

- ภาพรวม Collections
- users
- classes
- bookings
- checkins
- settings
- ความสัมพันธ์ระหว่างข้อมูล
- Index ที่จำเป็น
- หมายเหตุเรื่องชนิดข้อมูลวันที่

1. ภาพรวม Collections

Collection	Document ID	คำอธิบาย
users	LINE User ID	ข้อมูลสมาชิก โปรไฟล์ สิทธิ์ สถานะสมาชิก
classes	auto-id	คลาสออกกำลังกายแต่ละรอบ (วัน+เวลา)
bookings	auto-id	การจองคลาสของสมาชิก
checkins	auto-id	บันทึกการเช็คอินเข้าใช้บริการ
settings	ชื่อคองที่ (เช่น classTypes)	การตั้งค่าระบบ

2. Collection: `users`

Document ID = LINE User ID (เช่น `U1234567890abcdef ...`)

ฟิลด์	ชนิด	ความหมาย
lineUserId	string	LINE User ID (ตรงกับ doc id)
displayName	string	ชื่อที่แสดงจาก LINE
pictureUrl	string	URL รูปโปรไฟล์ LINE
nickname	string	ชื่อเล่น (กรอกตอนตั้งโปรไฟล์)
firstName	string	ชื่อจริง (ภาษาไทย)
lastName	string	นามสกุล (ภาษาไทย)
nationalId	string	เลขบัตรประชาชน 13 หลัก (validate checksum)
phone	string	เบอร์โทรศัพท์
birthDate	string	วันเกิด (yyyy-mm-dd)
gender	string	เพศ: male / female / other
healthIssues	string	ปัญหาสุขภาพ/การบาดเจ็บ
emergencyContact	object null	ผู้ติดต่อฉุกเฉิน { name, phone, relationship, hasContact }
role	string	บทบาท: user / staff / admin
memberType	string	แพ็คเกจ เช่น platinum , gold (gold จองไม่ได้)
membershipExpiry	Timestamp Date	วันหมดอายุสมาชิก
cooldownUntil	Timestamp null	วันสิ้นสุดการระงับสิทธิ์จอง
cooldownReason	string null	เหตุผลการระงับ
acceptTerms	boolean	ยอมรับข้อกำหนด
acceptRisk	boolean	ยอมรับความเสี่ยง
profileCompleted	boolean	ตั้งค่าโปรไฟล์เสร็จแล้ว
interests	array	ความสนใจ (อาจว่าง)
goals	string	เป้าหมายการออกกำลังกาย
yogaLevel	string	ระดับ เช่น beginner
createdAt	Date	วันที่สร้าง (ตอน import/สร้างใหม่)

ฟิลด์	ชนิด	ความหมาย
<code>updatedAt</code>	Date	วันที่แก้ไขล่าสุด

เขียนตอนสร้าง (ProfileSetup `completeProfileSetup` , Admin import): ฟิลด์โปรไฟล์ทั้งหมด + `profileCompleted=true` + `createdAt`

เขียนตอนแก้ไข:

- ผู้ใช้แก้ไขเอง: ฟิลด์โปรไฟล์ (ยกเว้น `role/membershipExpiry/cooldownUntil`)
- Admin: `membershipExpiry` , `memberType` , `role` , `cooldownUntil` , `cooldownReason` + `updatedAt`

3. Collection: `classes`

Document ID = auto-generated

ฟิลด์	ชนิด	ความหมาย
<code>type</code>	string	ประเภทหลัก: <code>yoga</code> / <code>dancing</code> / <code>wellness</code>
<code>subtype</code>	string	คลาสย่อย เช่น <code>flow-yoga</code> , <code>zumba</code> (ดู <code>classTypes.js</code>)
<code>name</code>	string	ชื่อคลาส (จาก label ของ subtype)
<code>description</code>	string	คำอธิบายคลาส (จาก subtype, แก้ไม่ได้)
<code>date</code>	string	วันที่จัด (yyyy-mm-dd)
<code>time</code>	string	เวลา (HH:mm)
<code>instructor</code>	string	ชื่อครูผู้สอน
<code>maxCapacity</code>	number	จำนวนที่นั่งสูงสุด (default 15)
<code>currentBookings</code>	number	จำนวนที่จองแล้ว (เริ่ม 0)
<code>createdAt</code>	Date	วันที่สร้าง
<code>updatedAt</code>	Date	วันที่แก้ไขล่าสุด (เฉพาะเมื่อแก้)

เขียนตอนสร้าง (Admin `addClass` , weekly script): ทุกฟิลด์ + `currentBookings=0` + `createdAt`

เขียนตอนแก้ไข (Admin `updateClass`): `type`, `subtype`, `name`, `description`, `date`, `time`, `instructor`, `maxCapacity`, `updatedAt`

อัปเดต counter (booking/cancel): `currentBookings` ด้วย `increment(±1)`

ประเภทคลาสและคลาสย่อย (จาก constants/classTypes.js)

type	คลาสย่อย (subtype)
yoga	Strength & Restoring, Hatha Vinyasa flow, Relaxing Recharge, Yoga Advance, Ashtanga, Power, Pilates Mat, Mixed Yoga, Flow Yoga, Chair Yoga
dancing	Easy Dance, Aerobic, Aero Boxing, Dance Burn, Zumba, Dance Variety, Piloxing
wellness	Sound bath

admin สามารถเพิ่ม/แก้ประเภทคลาสเองได้ผ่าน `settings/classTypes` (หับค่า default ใน `classTypes.js`)

4. Collection: bookings

Document ID = auto-generated

ฟิลด์	ชนิด	ความหมาย
userId	string	LINE User ID ของผู้จอง
classId	string	doc id ของคลาสที่จอง
className	string	ชื่อคลาส (denormalized)
date	string	วันที่คลาส (yyyy-mm-dd)
time	string	เวลาคลาส (HH:mm)
instructor	string	ครูผู้สอน (denormalized)
status	string	confirmed / cancelled / completed
bookedAt	Date	เวลาที่จอง
canCancelUntil	Date	เวลาที่ยกเลิกได้ถึง (= เวลาคลาสเริ่ม)
cancelledAt	Date	เวลาที่ยกเลิก (เมื่อ status=cancelled)

เขียนตอนสร้าง (Booking/ClassDetail transaction, Admin manual): userId, classId, className, date, time, instructor, status='confirmed', bookedAt, canCancelUntil

เขียนตอนยกเลิก (History): status='cancelled', cancelledAt

หมายเหตุ: ข้อมูล className/instructor ถูก denormalize เก็บไว้ใน booking เพื่อให้แสดงประวัติได้โดยไม่ต้อง join กับ classes (Firestore ไม่มี join)

5. Collection: `checkins`

Document ID = auto-generated

ฟิลด์	ชนิด	ความหมาย
<code>uid</code>	string	LINE User ID ของผู้เช็คอิน
<code>name</code>	string	ชื่อสมาชิก (nickname/displayName)
<code>memberType</code>	string	แพ็คเกจสมาชิก
<code>membershipValid</code>	boolean	สมาชิกยังไม่หมดอายุ ณ เวลาเช็คอิน
<code>classId</code>	string	คลาสที่เช็คอิน (ว่างได้)
<code>className</code>	string	ชื่อคลาส
<code>classDate</code>	string	วันที่คลาส
<code>classTime</code>	string	เวลาคลาส
<code>instructor</code>	string	ครูผู้สอน
<code>checkedInAt</code>	Timestamp	เวลาที่เช็คอิน

เขียนตอนสร้าง (Admin/CheckinScanner): ทุกฟิลด์ข้างต้น **ลบ** (Admin `deleteCheckin`): ลบทั้ง document ได้ (ฝั่งแอป)

หมายเหตุ: Security Rules กำหนด `allow update, delete: if false` สำหรับ checkins แต่โค้ด Admin มี `deleteCheckin` — จะใช้งานได้ต่อเมื่อ rules ปัจจุบันยังไม่ถูก deploy หรือ deploy ในเวอร์ชันที่อนุญาต

6. Collection: `settings`

Document ID = ชื่อคองที่

Document	เนื้อหา	ความหมาย
<code>classTypes</code>	object	การตั้งค่าประเภท/คลาสย่อยที่ admin กำหนดเอง (ทับ default)

อ่าน (Admin `loadCustomClassSettings`): `getDoc(settings/classTypes)` เขียน (Admin เพิ่ม/แก้/ลบประเภทคลาส): `setDoc(settings/classTypes)`

7. ความสัมพันธ์ระหว่างข้อมูล

```

users (1) —< (N) bookings      [bookings.userId → users.id]
classes (1) —< (N) bookings     [bookings.classId → classes.id]
users (1) —< (N) checkins       [checkins.uid → users.id]
classes (1) —< (N) checkins     [checkins.classId → classes.id]
settings/classTypes —> นิยามประเภทของ classes.type/subtype

```

- ความสัมพันธ์เป็นแบบ **reference by ID** (ไม่มี foreign key จริง — Firestore ไม่บังคับ referential integrity)
- ข้อมูลบางส่วน denormalize (className, instructor ใน bookings/checkins) เพื่อลดการอ่านข้าม collection

8. Index ที่จำเป็น

Firestore ต้องการ composite index สำหรับ query ที่กรอง/เรียงหลายฟิลด์:

Collection	Query	ฟิลด์ index
<code>classes</code>	ตามวันที่ (Booking)	<code>date</code> (single, อัปเดตอัตโนมัติ)
<code>classes</code>	ช่วงวันที่ (weekly script)	<code>date</code> range (single)
<code>classes</code>	ตรวจซ้ำ (weekly script)	<code>date</code> + <code>time</code> + <code>name</code> (composite)
<code>bookings</code>	ของผู้ใช้ (History/Booking)	<code>userId</code> (single, อัปเดตอัตโนมัติ)

query แบบ `where('date','==', ...)` และ `where('userId','==', ...)` ใช้ single-field index ที่ Firestore สร้างให้อัตโนมัติ query ในสคริปต์ที่กรอง 3 ฟิลด์ (`date` + `time` + `name`) ต้องสร้าง composite index ใน Firebase Console หากยังไม่มี

9. หมายเหตุเรื่องชนิดข้อมูลวันที่

ระบบใช้ชนิดข้อมูลที่ปนกันหลายแบบ ควรระวัง:

- **string (yyyy-mm-dd):** `date` ใน classes/bookings — ใช้ string เพื่อ query แบบ exact/range ได้ง่าย
- **Date (JavaScript):** เขียนตรงๆ เช่น `bookedAt: new Date()` Firestore SDK แปลงเป็น Timestamp อัตโนมัติ
- **Timestamp (Firestore):** `membershipExpiry` , `cooldownUntil` , `checkedInAt` ใช้ `Timestamp.fromDate()` — ต้องเรียก `.toDate()` ตอนอ่าน

โค้ดมีการ normalize หลายจุด เช่น `auth.js` แปลง `membershipExpiry.toDate()` และตรวจทั้งกรณี Timestamp/string/Date ใน `isMembershipValid()`

ข้อแนะนำ: ควรกำหนดมาตรฐานชนิดข้อมูลวันที่ให้สม่ำเสมอ (เช่น Timestamp ทั้งหมดสำหรับ datetime, string yyyy-mm-dd สำหรับวันที่ปฏิทิน) เพื่อลด bug

เอกสารนี้สร้างจากการวิเคราะห์ซอร์สโค้ดจริง ณ เวอร์ชันปัจจุบัน